

UNIVERSIDADE FEDERAL DE SERGIPE

Centro de Ciências Exatas e Tecnologia

Departamento de Computação

**AVALIAÇÃO DE DESEMPENHO NA TRANSFERÊNCIA DE
ARQUIVOS: CLIENTE-SERVIDOR × P2P**

Aluno: Bruno Henrique Carneiro da Silva

Professor: Rafael Oliveira Vasconcelos

São Cristóvão – 2026

1. INTRODUÇÃO

A transferência de arquivos é uma operação comum em sistemas distribuídos e pode ser implementada por meio de diferentes arquiteturas de comunicação. Entre as abordagens utilizadas, destacam-se o modelo cliente-servidor e o modelo ponto a ponto (Peer-to-Peer ou P2P), que apresentam diferenças na organização das conexões, na distribuição dos dados e no gerenciamento dos recursos computacionais.

No modelo cliente-servidor, os clientes solicitam os arquivos a um servidor central, que é responsável por disponibilizar os dados. Essa abordagem pode ser implementada de diferentes formas, como no atendimento sequencial das conexões, na criação de uma thread para cada cliente ou na utilização de um conjunto limitado de threads. Cada estratégia apresenta características distintas em relação ao paralelismo e à concorrência no atendimento das solicitações.

Já no modelo P2P, os participantes podem atuar simultaneamente como consumidores e fornecedores de partes do arquivo. Dessa forma, os dados podem ser distribuídos entre diferentes peers, reduzindo a dependência de um único servidor durante a transferência. Entretanto, o desempenho dessa arquitetura depende de fatores como a implementação do protocolo, a quantidade de participantes e o custo de coordenação e armazenamento dos dados.

Neste trabalho, foram implementadas e avaliadas quatro abordagens para a transferência de arquivos: cliente-servidor sequencial, cliente-servidor com uma thread por cliente, cliente-servidor com pool de threads e P2P. Os experimentos consideraram diferentes tamanhos de arquivos e quantidades de clientes, permitindo observar o comportamento das implementações sob diferentes condições de concorrência.

2. OBJETIVO

O objetivo deste trabalho é comparar o desempenho de quatro arquiteturas de transferência de arquivos, analisando o tempo necessário para a conclusão das transferências em diferentes condições de execução.

Foram considerados arquivos de 5 MB, 50 MB e 500 MB, com 1, 5, 10 e 20 clientes. Para cada combinação, foram registrados os tempos mínimo, médio e máximo de conclusão, possibilitando observar as diferenças entre as abordagens e identificar como o aumento do volume de dados e da quantidade de participantes influencia o desempenho.

3. METODOLOGIA

3.1. Tecnologias utilizadas

As implementações foram desenvolvidas em Python 3.13, utilizando sockets TCP para estabelecer a comunicação entre os processos. Os testes foram realizados com arquivos de diferentes tamanhos e com quantidades variadas de clientes, de acordo com os cenários definidos para a atividade.

Foram implementadas quatro arquiteturas: atendimento sequencial, atendimento com uma thread por cliente, atendimento com pool de threads e transferência no modelo P2P.

3.2. Implementação das arquiteturas

3.2.1. Cliente-servidor sequencial

Na implementação sequencial, o servidor atende um cliente por vez. Enquanto uma transferência está sendo realizada, os demais clientes aguardam na fila de conexões até que o servidor esteja disponível para atendê-los.

Essa abordagem apresenta uma estrutura simples, mas o tempo total de atendimento pode aumentar conforme cresce a quantidade de clientes, uma vez que as transferências não são processadas simultaneamente pelo servidor.

3.2.2. Cliente-servidor com uma thread por cliente

Nessa arquitetura, o servidor cria uma nova thread para atender cada conexão recebida. Dessa forma, vários clientes podem realizar transferências simultaneamente, sem que seja necessário aguardar a conclusão de todas as transferências anteriores.

Embora esse modelo permita maior concorrência, o aumento da quantidade de clientes também pode elevar o consumo de recursos e o custo de gerenciamento das threads.

3.2.3. Cliente-servidor com pool de threads

Na implementação com pool de threads, foi utilizado um conjunto limitado a quatro threads. Assim, no máximo quatro transferências são processadas simultaneamente, enquanto os demais clientes aguardam até que uma das threads esteja disponível.

Essa estratégia busca controlar o uso de recursos computacionais e limitar a concorrência no servidor. Seu desempenho depende da relação entre o número de clientes, o tamanho dos arquivos e a capacidade de processamento disponível.

3.2.4. Arquitetura P2P

A implementação P2P foi desenvolvida com base em uma abordagem simplificada inspirada no funcionamento do BitTorrent. O arquivo é dividido em blocos de 256 KB, permitindo que diferentes partes sejam transferidas entre os participantes.

Inicialmente, existe um peer que possui o arquivo completo, denominado seed. Os demais peers solicitam os blocos necessários e podem compartilhar com outros participantes os blocos que já receberam. A solicitação dos blocos ocorre em ordem aleatória, e as transferências podem acontecer em paralelo.

Os blocos recebidos são armazenados temporariamente em arquivos, permitindo que sejam compartilhados com outros peers durante a execução. Ao final da transferência, os arquivos temporários são removidos.

Essa implementação permite observar o comportamento de uma estratégia de distribuição descentralizada, embora não corresponda a uma implementação completa do protocolo BitTorrent.

3.3. Procedimento de medição

Nos três modelos cliente-servidor, o tempo de transferência de cada cliente foi medido desde o início da conexão até o recebimento do último byte do arquivo. O tempo inclui, portanto, a espera na fila de conexões, quando existente.

Na implementação P2P, a medição considera o intervalo entre o início sincronizado dos participantes e o momento em que cada peer possui todos os blocos necessários para reconstruir o arquivo.

Os clientes e peers foram iniciados de forma simultânea em cada cenário. Para os modelos cliente-servidor, os dados recebidos foram descartados, sem gravação do arquivo completo em disco. Na implementação P2P, os blocos foram gravados temporariamente para possibilitar o compartilhamento entre os participantes.

3.4. Cenários experimentais

Foram utilizados arquivos de 5 MB, 50 MB e 500 MB, com 1, 5, 10 e 20 clientes. No cenário P2P, além dos peers participantes do experimento, foi utilizado um seed inicial contendo o arquivo completo.

Cada combinação de tamanho de arquivo, quantidade de clientes e arquitetura foi executada repetidamente. Em geral, foram realizadas seis repetições por combinação, com variações entre três e sete execuções.

Em cada execução, foram registrados os tempos mínimo, médio e máximo de conclusão entre os clientes. Os valores apresentados nas tabelas correspondem às médias desses indicadores ao longo das repetições realizadas.

3.5. Ambiente de execução

Todos os experimentos foram executados em uma única máquina com sistema operacional Windows e Python 3.13. O servidor, os clientes e os peers foram executados localmente, utilizando a interface de loopback, no endereço 127.0.0.1.

Consequentemente, os testes não envolveram a comunicação entre computadores distintos por meio de uma rede física. Essa característica deve ser considerada na interpretação dos resultados, pois o desempenho observado também é influenciado pelo processamento local, pelo sistema operacional e pelos mecanismos de comunicação entre processos.

4. RESULTADOS EXPERIMENTAIS

Nesta seção, são apresentados os tempos de conclusão das transferências para cada arquitetura, considerando os três tamanhos de arquivo e as quatro quantidades de clientes avaliadas. Os valores estão expressos em segundos e representam o tempo mínimo, médio e máximo entre os clientes, conforme o procedimento de medição descrito anteriormente.

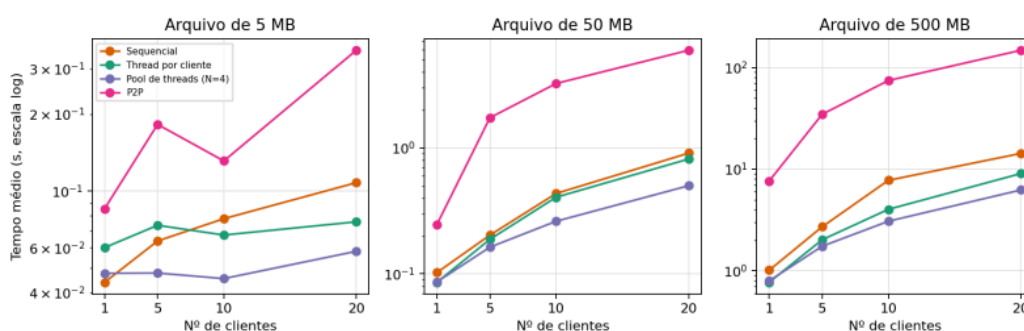


Figura 1 – Tempo médio de conclusão em função do número de clientes (escala logarítmica).

4.1. Transferência de arquivo de 5 MB

Tabela 1 – Tempos de transferência para arquivo de 5 MB

Clientes	Arquitetura	Mínimo (s)	Médio (s)	Máximo (s)
1	Sequencial	0,04	0,04	0,04
	Thread por cliente	0,06	0,06	0,06
	Pool de threads (N=4)	0,05	0,05	0,05
	P2P	0,09	0,09	0,09
5	Sequencial	0,03	0,06	0,10
	Thread por cliente	0,05	0,07	0,10
	Pool de threads (N=4)	0,04	0,05	0,06
	P2P	0,16	0,18	0,20
10	Sequencial	0,02	0,08	0,13
	Thread por cliente	0,06	0,07	0,08
	Pool de threads (N=4)	0,03	0,05	0,06
	P2P	0,10	0,13	0,16
20	Sequencial	0,02	0,11	0,20
	Thread por cliente	0,05	0,08	0,10
	Pool de threads (N=4)	0,02	0,06	0,09
	P2P	0,33	0,35	0,36

Análise dos resultados para 5 MB

Para o arquivo de 5 MB, os tempos registrados foram relativamente baixos em todas as arquiteturas, principalmente devido ao pequeno volume de dados transferido. Com apenas um cliente, o modelo sequencial apresentou tempo médio de 0,04 segundo, enquanto o modelo com thread por cliente apresentou 0,06 segundo, o pool de threads apresentou 0,05 segundo e o P2P apresentou 0,09 segundo.

Com o aumento da quantidade de clientes, observa-se que o tempo médio do modelo sequencial aumentou de 0,04 segundo, com um cliente, para 0,11 segundo, com 20 clientes. O modelo com thread por cliente apresentou tempo médio de 0,08 segundo no cenário com 20 clientes, enquanto o pool de threads registrou 0,06 segundo.

A implementação P2P apresentou tempo médio de 0,35 segundo com 20 peers, superior aos valores observados nas demais arquiteturas. Nesse cenário, o custo de dividir o arquivo em blocos, coordenar as solicitações e armazenar temporariamente os dados pode ter representado uma parcela significativa do tempo total, especialmente porque o arquivo utilizado era pequeno.

4.2. Transferência de arquivo de 50 MB

Tabela 2 – Tempos de transferência para arquivo de 50 MB

Clientes	Arquitetura	Mínimo (s)	Médio (s)	Máximo (s)
1	Sequencial	0,10	0,10	0,10
	Thread por cliente	0,09	0,09	0,09
	Pool de threads (N=4)	0,09	0,09	0,09
	P2P	0,25	0,25	0,25
5	Sequencial	0,08	0,20	0,32
	Thread por cliente	0,18	0,19	0,19
	Pool de threads (N=4)	0,15	0,16	0,22
	P2P	1,67	1,74	1,78
10	Sequencial	0,14	0,43	0,72
	Thread por cliente	0,38	0,41	0,42
	Pool de threads (N=4)	0,15	0,26	0,41
	P2P	2,25	3,25	4,07
20	Sequencial	0,21	0,91	1,53
	Thread por cliente	0,74	0,82	0,86
	Pool de threads (N=4)	0,16	0,50	0,86
	P2P	0,45	5,96	9,25

Análise dos resultados para 50 MB

No cenário com arquivos de 50 MB, os efeitos do aumento da quantidade de clientes tornam-se mais evidentes. O modelo sequencial apresentou crescimento do tempo médio de 0,10 segundo, com um cliente, para 0,91 segundo, com 20 clientes. Esse comportamento é compatível com a necessidade de atender sucessivamente às conexões recebidas.

A arquitetura com uma thread por cliente apresentou tempo médio de 0,82 segundo no cenário com 20 clientes. Embora permita transferências simultâneas, seu desempenho também pode ser afetado pelo gerenciamento das threads e pela disputa por recursos compartilhados.

O pool de threads apresentou tempo médio de 0,50 segundo com 20 clientes. Como o número de threads foi limitado a quatro, a implementação controlou a quantidade de transferências simultâneas, apresentando um resultado médio inferior ao das outras duas abordagens cliente-servidor nesse cenário.

O P2P apresentou tempo médio de 5,96 segundos com 20 peers, além de uma diferença considerável entre o mínimo registrado, de 0,45 segundo, e o máximo, de 9,25 segundos. Essa variação indica que os resultados desse cenário apresentaram dispersão elevada entre as execuções. Portanto, a média deve ser interpretada em conjunto com os valores mínimo e máximo, e não de forma isolada.

4.3. Transferência de arquivo de 500 MB

Tabela 3 – Tempos de transferência para arquivo de 500 MB

Clientes	Arquitetura	Mínimo (s)	Médio (s)	Máximo (s)
1	Sequencial	1,01	1,01	1,01
	Thread por cliente	0,77	0,77	0,77
	Pool de threads (N=4)	0,80	0,80	0,80
	P2P	7,64	7,64	7,64
5	Sequencial	1,37	2,71	4,00
	Thread por cliente	1,99	2,02	2,05
	Pool de threads (N=4)	1,60	1,74	2,26
	P2P	32,31	34,65	36,09
10	Sequencial	4,24	7,76	11,07
	Thread por cliente	3,78	4,02	4,20
	Pool de threads (N=4)	1,88	3,07	4,56
	P2P	63,41	74,53	82,48
20	Sequencial	4,44	14,28	23,65
	Thread por cliente	7,85	9,05	9,90
	Pool de threads (N=4)	2,18	6,23	10,37
	P2P	106,85	147,84	175,60

Análise dos resultados para 500 MB

O arquivo de 500 MB apresentou os maiores tempos de transferência entre os cenários avaliados. O aumento do volume de dados tornou mais perceptíveis as diferenças de desempenho entre as arquiteturas, principalmente quando a quantidade de clientes foi ampliada.

Com apenas um cliente, a implementação com uma thread por cliente registrou tempo médio de 0,77 segundo, enquanto o pool de threads apresentou 0,80 segundo e o modelo sequencial, 1,01 segundo. A implementação P2P apresentou tempo médio de 7,64 segundos nesse mesmo cenário.

Com cinco clientes, o modelo sequencial apresentou tempo médio de 2,71 segundos, enquanto o modelo com thread por cliente registrou 2,02 segundos e o pool de threads, 1,74 segundo. O P2P apresentou tempo médio de 34,65 segundos.

No cenário com dez clientes, o pool de threads registrou tempo médio de 3,07 segundos, inferior aos 4,02 segundos do modelo com thread por cliente e aos 7,76 segundos do modelo sequencial. O P2P apresentou tempo médio de 74,53 segundos.

Com 20 clientes, o pool de threads apresentou tempo médio de 6,23 segundos, enquanto o modelo com thread por cliente registrou 9,05 segundos e o modelo sequencial, 14,28 segundos. A implementação P2P apresentou tempo médio de 147,84 segundos, com mínimo de 106,85 segundos e máximo de 175,60 segundos.

Nesse cenário, os resultados sugerem que o pool de threads conseguiu controlar a concorrência de maneira favorável à execução local dos testes. Entretanto, essa

observação deve ser restrita ao ambiente e à implementação utilizados. Como todos os processos foram executados na mesma máquina, a disputa por processamento, memória, armazenamento e recursos de comunicação pode ter influenciado significativamente os resultados.

A implementação P2P apresentou tempos maiores nos cenários testados, especialmente com arquivos grandes e muitos peers. Esse comportamento pode estar relacionado ao custo adicional de dividir o arquivo em blocos, coordenar as transferências, gravar dados temporários e compartilhar os blocos entre os participantes. Esses resultados não significam, por si só, que o modelo P2P seja necessariamente inferior em ambientes distribuídos reais, nos quais os peers podem contribuir com recursos de rede independentes.



Figura 2 – Variações cliente-servidor com 500 MB e 20 clientes: mínimo, médio e máximo.

5. DISCUSSÃO GERAL

A comparação entre as quatro arquiteturas evidencia que o desempenho de uma transferência depende tanto do volume de dados quanto da forma como as conexões e os recursos computacionais são gerenciados.

A implementação sequencial apresentou um comportamento simples e previsível, mas o aumento do número de clientes elevou o tempo médio de conclusão em vários cenários. Isso ocorre porque o atendimento é realizado individualmente, fazendo com que clientes posteriores possam aguardar a conclusão das transferências anteriores.

A implementação com uma thread por cliente permitiu atender várias conexões simultaneamente. Entretanto, a criação e o gerenciamento de um número crescente de threads podem aumentar o consumo de recursos. Assim, embora essa abordagem tenha apresentado bons resultados em alguns cenários, seu desempenho não foi necessariamente o melhor em todas as combinações testadas.

O pool de threads, limitado a quatro threads, apresentou tempos médios inferiores aos das outras implementações cliente-servidor em alguns cenários com maior número de clientes. No caso do arquivo de 500 MB com 20 clientes, por exemplo, o tempo médio foi de 6,23 segundos, comparado a 9,05 segundos no modelo com thread por cliente e 14,28 segundos no sequencial. Esse resultado indica que limitar a concorrência pode ser vantajoso quando o número de solicitações é elevado, embora a configuração ideal do pool dependa do ambiente e das características da carga de trabalho.

A implementação P2P apresentou tempos superiores aos das demais arquiteturas nos experimentos realizados. Contudo, sua estrutura possui características diferentes das abordagens cliente-servidor, pois permite que os participantes compartilhem os blocos já recebidos. A avaliação dessa vantagem exige considerar o ambiente de execução. Como os testes foram realizados localmente, não foi possível observar diretamente os benefícios da distribuição dos dados entre computadores com conexões de rede independentes.

Outro aspecto relevante é a variação entre os tempos mínimo e máximo. Em alguns cenários, especialmente na implementação P2P com 20 peers, a diferença entre os valores foi elevada. Isso demonstra a importância de analisar medidas de dispersão e de realizar múltiplas execuções, em vez de utilizar apenas um único resultado para comparar as arquiteturas.

5.1. Limitações do experimento

Os resultados devem ser interpretados considerando algumas limitações metodológicas. A principal delas é que todos os processos foram executados em uma única máquina, utilizando a interface de loopback. Portanto, os experimentos não reproduzem integralmente as condições de uma rede distribuída real, na qual os participantes podem estar em computadores distintos e apresentar latências, larguras de banda e capacidades de processamento diferentes.

Além disso, as implementações possuem características próprias que influenciam os tempos medidos. Nos modelos cliente-servidor, os dados recebidos foram descartados, enquanto na implementação P2P os blocos foram gravados temporariamente para possibilitar o compartilhamento. Essa diferença de comportamento deve ser considerada ao comparar os valores obtidos.

A quantidade de repetições também variou entre as combinações, ficando geralmente em torno de seis execuções, com variações entre três e sete. Por isso, os resultados constituem uma avaliação experimental das implementações desenvolvidas, e não uma medida universal do desempenho de cada arquitetura.

Por fim, o sistema P2P utilizado é uma implementação simplificada inspirada no BitTorrent. Os resultados refletem o funcionamento dessa implementação

específica, incluindo seus mecanismos de coordenação, transferência e armazenamento temporário, não podendo ser generalizados para todas as soluções P2P.

6. CONCLUSÃO

Este trabalho apresentou uma comparação experimental entre quatro arquiteturas de transferência de arquivos: cliente-servidor sequencial, cliente-servidor com uma thread por cliente, cliente-servidor com pool de threads e P2P. Os testes consideraram arquivos de 5 MB, 50 MB e 500 MB, com 1, 5, 10 e 20 clientes, registrando os tempos mínimo, médio e máximo de conclusão.

Os resultados demonstraram que o aumento do tamanho dos arquivos e da quantidade de clientes pode alterar significativamente o desempenho das implementações. O modelo sequencial apresentou aumento dos tempos médios em vários cenários com maior concorrência, enquanto a criação de uma thread por cliente permitiu o atendimento simultâneo das conexões, com custos associados ao gerenciamento das threads.

O pool de threads apresentou resultados favoráveis em alguns cenários de maior concorrência. Para o arquivo de 500 MB com 20 clientes, por exemplo, a média observada foi de 6,23 segundos, inferior aos 9,05 segundos da implementação com thread por cliente e aos 14,28 segundos da implementação sequencial.

A implementação P2P apresentou tempos médios maiores nos cenários avaliados, especialmente para arquivos grandes e quantidades elevadas de peers. Entretanto, como os experimentos foram executados em uma única máquina, os resultados não permitem concluir que essa arquitetura seria necessariamente menos eficiente em uma rede real. O custo de coordenação e armazenamento da implementação utilizada, assim como as características do ambiente local, influenciou os valores obtidos.

Conclui-se que não existe uma única estratégia cujo desempenho possa ser considerado superior em todas as condições com base apenas nos experimentos realizados. A escolha de uma arquitetura deve considerar a quantidade de clientes, o tamanho dos arquivos, os recursos disponíveis e as características da rede. Como possibilidade de continuidade, os testes poderiam ser repetidos em computadores distintos, com diferentes condições de rede e com uma análise mais detalhada da variabilidade dos resultados.

7. REFERÊNCIAS E RECURSOS DO PROJETO

Linguagem de programação: Python 3.13.

Tecnologia de comunicação: sockets TCP.

Repositório do projeto:

<https://github.com/brunohenriquedeveloper/Atividade-01-P2P---Sistemas-Distribuidos>.

Documentação: README.md disponibilizado no repositório do projeto.